Dokumentacija projekta

Portanje aplikacije TetriSaraj na operativni sistem FreeRTOS na platformi RISC-V (Tang Nano 9K / PicoRV32)

Autor: Delaida Muminović

Zenica, 2026.

Sažetak

U radu se razmatra portanje postojeće aplikacije TetriSaraj sa golog mikrokontrolerskog izvršavanja na operativni sistem za rad u realnom vremenu FreeRTOS, na otvorenoj RISC-V platformi. Ciljna platforma je meki procesor PicoRV32 sintetizovan na razvojnoj ploči Tang Nano 9K. Prikazuje se značaj operativnih sistema za rad u realnom vremenu u kontekstu ugradbenih sistema sa ograničenim resursima, gdje je potrebno uskladiti vremensku predvidljivost, zauzeće memorije i logičkih resursa programabilne logike. Cilj rada je objasniti postupak prevođenja monolitne aplikacije u skup zadataka pod upravom raspoređivača, kao i postupak uklapanja takvog sistema zajedno sa video izlazom u stroge resurse niskobudžetne ploče. Metode korištene pri izradi rada obuhvataju razvoj i prevođenje ugradbenog programa, sintezu digitalnog sklopa, mjerenje na stvarnom hardveru putem serijske veze i poređenje sa rezultatima dobijenim u simulatoru.

Ključne riječi: FreeRTOS, PicoRV32, RISC-V, sinteza, Tang Nano 9K, ugradbeni sistemi, zamjena konteksta.

Abstract

This paper considers porting the existing TetriSaraj application from bare-metal execution to the FreeRTOS real-time operating system on an open RISC-V platform. The target platform is the PicoRV32 soft processor synthesised on a Tang Nano 9K development board. The importance of real-time operating systems is presented in the context of resource-constrained embedded systems, where timing predictability, memory usage and programmable-logic resource usage must be balanced. The aim of the paper is to explain the process of converting a monolithic application into a set of scheduler-managed tasks, as well as the process of fitting such a system together with video output into the tight resources of a low-cost board. The methods used in the work include embedded firmware development and compilation, digital-circuit synthesis, measurement on real hardware over a serial link, and comparison with results obtained in a simulator.

Keywords: context switch, embedded systems, FreeRTOS, PicoRV32, RISC-V, synthesis, Tang Nano 9K.

Sadržaj

Sažetak **1**

Abstract **2**

Uvod **5**

1 Srodni radovi **7**
1.1 Mjerenje performansi operativnih sistema za rad u realnom vremenu 7
1.2 Zamjena konteksta i prekidi na arhitekturi RISC-V 7
1.3 Meki procesori RISC-V na programabilnoj logici 8
1.4 Odnos simulacije i stvarnog hardvera 8
1.5 Pozicija ovog rada 8
1.6 Zaključak poglavlja 9

2 Hardverska platforma i FreeRTOS port **10**
2.1 Tang Nano 9K i meki procesor PicoRV32 10
2.2 Prilagođeni port operativnog sistema FreeRTOS 11
2.3 Memorijska i adresna organizacija sistema na čipu 11
2.4 Dokaz pokretanja na hardveru 12
2.5 Zaključak poglavlja 12

3 Razdvajanje aplikacije na zadatke **13**
3.1 Razdvajanje logike igre i pristupa hardveru 13
3.2 Arhitektura zadataka i komunikacija 13
3.3 Dokaz rada na hardveru bez video izlaza 14
3.4 Zaključak poglavlja 15

4 Uklapanje cijele igre sa video izlazom **16**
4.1 Problem uklapanja 16
4.2 Analiza zauzeća blokovske memorije 16
4.3 Dva rješenja: manji program i manja video memorija 17

4.4 Rezultat uklapanja i prilagodba video inicijalizacije 17
4.5 Zaključak poglavlja 18

5 Mjerenje zamjene konteksta i poređenje sa simulatorom 19
5.1 Metodologija mjerenja 19
5.2 Rezultati na hardveru 20
5.3 Poređenje sa simulatorom 20
5.4 Zaključak poglavlja 21

6 Pravci daljnjeg istraživanja 22
6.1 Kvantifikacija troška operativnog sistema 22
6.2 Usklađivanje metrike i statistička obrada 22
6.3 Uticaj mikroarhitekture i skupa instrukcija 23
6.4 Latencija i determinizam prekida pod opterećenjem 23
6.5 Sistematsko poređenje simulatora i hardvera 23
6.6 Hardversko ubrzanje raspoređivanja 23
6.7 Zaključak poglavlja 23

Zaključak 25

Popis slika 26

Literatura 27

Uvod

Ugradbeni sistemi danas čine najveći dio računarskih uređaja u svakodnevnoj upotrebi, od kućnih aparata do industrijske opreme. Veliki broj takvih sistema ima zahtjeve za vremenskom predvidljivošću, pa se za njihovo upravljanje koriste operativni sistemi za rad u realnom vremenu. Jedan od najčešće korištenih takvih sistema otvorenog koda je FreeRTOS, koji je dostupan za veliki broj arhitektura procesora. Istovremeno se sve više koristi otvorena skupovna arhitektura RISC-V, koja omogućava izradu vlastitih procesorskih jezgara bez licencnih ograničenja. Spajanje ova dva otvorena pristupa, operativnog sistema otvorenog koda i otvorene procesorske arhitekture, postaje zanimljiva i aktuelna tema, posebno na jeftinim razvojnim pločama sa programabilnom logikom.

Razvojna ploča Tang Nano 9K, zasnovana na programabilnom logičkom polju (*Field Programmable Gate Array*, FPGA), pruža pristupačan način da se na njoj sintetizuje meki procesor i izvršava stvarni program. Meki procesor PicoRV32 jednostavna je RISC-V jezgra pogodna za takve eksperimente. Međutim, resursi ovakvih ploča su veoma ograničeni, prije svega blokovska memorija i količina logičkih elemenata, pa uklapanje složenijih sistema postaje inženjerski izazov. Upravo to čini portanje cijele aplikacije zajedno sa operativnim sistemom i video izlazom netrivijalnim zadatkom.

Ovaj rad nastavlja se na postojeću aplikaciju TetriSaraj, koja je izvorno pisana kao monolitni program bez operativnog sistema. Postavljaju se sljedeća pitanja na koja rad treba odgovoriti. Da li se aplikacija može prevesti u skup zasebnih zadataka pod upravom raspoređivača operativnog sistema FreeRTOS i dokazati njen rad na stvarnom hardveru? Da li se takav sistem, zajedno sa video izlazom preko digitalnog multimedijalnog sučelja (*High-Definition Multimedia Interface*, HDMI), može uklopiti u stroge resurse ploče Tang Nano 9K? Te konačno, kolika je cijena zamjene konteksta na stvarnom hardveru i kako se ona odnosi prema ranije dobijenim vrijednostima iz simulatora? Pretpostavlja se da je sve to moguće uz pažljivo upravljanje memorijom i resursima.

Rad je organizovan na sljedeći način. U prvom poglavlju daje se pregled

srodnih radova i smješta vlastiti rad u kontekst postojeće literature. Drugo poglavlje opisuje hardversku platformu i prilagođeni port operativnog sistema FreeRTOS na meki procesor, te osnovni dokaz njegovog rada. Treće poglavlje prikazuje postupak razdvajanja aplikacije na zadatke i dokaz njihovog rada na hardveru bez video izlaza. Četvrto poglavlje objašnjava problem zauzeća resursa pri uklapanju cijele igre sa video izlazom i rješenja koja su to omogućila. Peto poglavlje opisuje mjerenje vremena zamjene konteksta na hardveru i poređenje sa simulatorom. Šesto poglavlje izdvaja moguće pravce daljnjeg istraživanja. Na kraju rada dat je zaključak u kojem su sistematizirani najvažniji rezultati i doprinosi rada. Nakon zaključka nalazi se popis slika i lista korištenih bibliografskih referenci.

Poglavlje 1

Srodni radovi

U ovom poglavlju daje se pregled dosadašnjih istraživanja koja su bliska temi rada. Cilj je smjestiti vlastiti rad u širi kontekst i jasno pokazati čime se on razlikuje od postojeće literature. Najprije se razmatraju radovi o mjerenju performansi operativnih sistema za rad u realnom vremenu, zatim radovi o zamjeni konteksta i prekidima na arhitekturi RISC-V, potom radovi o mekim procesorima na programabilnoj logici, te na kraju radovi o odnosu simulacije i stvarnog hardvera. Na osnovu tog pregleda izdvaja se pozicija ovog rada.

1.1 Mjerenje performansi operativnih sistema za rad u realnom vremenu

Mjerenje vremena zamjene konteksta uobičajena je metrika pri poređenju operativnih sistema za rad u realnom vremenu. Poređenja sistema FreeRTOS i Zephyr pokazuju da se ovakve veličine izražavaju u broju takta, pri čemu se za FreeRTOS navodi red veličine od oko dvije stotine dvadeset tri takta na jezgrama tipa ARM Cortex-M [1]. Važno je naglasiti da takve vrijednosti mjere užu operaciju zamjene konteksta na cjevovodnom procesoru, dok mjerenja u ovom radu obuhvataju i dodatni trošak rada sa mehanizmom sinhronizacije na višeciklusnom mekom procesoru, pa se brojevi ne mogu direktno poređivati bez te ograde.

1.2 Zamjena konteksta i prekidi na arhitekturi RISC-V

Poseban pravac istraživanja bavi se ubrzanjem zamjene konteksta i obrade prekida na arhitekturi RISC-V. U radu o proširenju CV32RT predlažu se

mikroarhitekturna proširenja jezgre radi bržeg ulaska u prekid i brže zamjene konteksta za mikrokontrolere zasnovane na RISC-V [2]. Slično, postoji istraživanje koje zajednički istražuje mikroarhitekturu procesora i proširenja samog operativnog sistema FreeRTOS sa ciljem smanjenja latencije zamjene konteksta [3]. Ovi radovi pokazuju da je smanjenje cijene zamjene konteksta aktuelna tema, ali se oni uglavnom oslanjaju na izmjene hardvera ili jezgre, dok se ovaj rad bavi mjerenjem nepromijenjenog sistema na postojećoj mekoj jezgri.

1.3 Meki procesori RISC-V na programabilnoj logici

Meki procesori na programabilnoj logici predmet su brojnih evaluacija. Procesor PicoRV32, korišten i u ovom radu, ocijenjen je kao izrazito štedljiv po pitanju resursa i pogodan za uređaje sa ograničenim resursima [4]. Takve evaluacije obično se fokusiraju na zauzeće logičkih elemenata i propusnost, dok je rad operativnog sistema na takvoj jezgri i njegovo uklapanje zajedno sa video izlazom rjeđe obrađivano. Upravo to čini doprinos ovog rada, jer se uz mjerenje razmatra i praktično ograničenje blokovske memorije pri sintezi.

1.4 Odnos simulacije i stvarnog hardvera

Emulator QEMU široko se koristi za pokretanje ugradbenog softvera, ali je on funkcijski tačan, što znači da vjerno oponaša ponašanje instrukcija, ali ne modeluje mikroarhitekturu takovno tačno [5]. Zbog toga se vrijednosti takta dobijene u takvom okruženju ne mogu bez ograde tumačiti kao vrijednosti na stvarnom čipu. Ova osobina direktno opravdava opreznost pri poređenju rezultata iz simulatora [6] sa rezultatima izmjerenim na hardveru, kako je učinjeno u poglavlju 5.

1.5 Pozicija ovog rada

Iz pregleda literature može se uočiti obrazac. Mjerenja performansi operativnih sistema najčešće se rade na cjevovodnim mikrokontrolerima tipa ARM Cortex-M, dok se na arhitekturi RISC-V težište stavlja na hardversko ubrzanje zamjene konteksta. Evaluacije mekih procesora fokusiraju se na resurse, a rjeđe na rad operativnog sistema. Ovaj rad popunjava prazninu time što mjeri nepromijenjeni FreeRTOS na minimalnoj otvorenoj mekoj

RISC-V jezgri na jeftinoj ploči, upoređuje rezultat sa simulatorom i dokumentuje praktično ograničenje sinteze vezano za blokovsku memoriju. Time se doprinos rada jasno razlikuje od navedenih istraživanja.

1.6 Zaključak poglavlja

U poglavlju je dat pregled srodnih istraživanja iz četiri bliske oblasti i izdvojena je pozicija ovog rada. Pokazano je da postojeća literatura uglavnom mjeri operativne sisteme na cjevovodnim jezgrama ili predlaže hardverska ubrzanja, dok mjerenje nepromijenjenog sistema na minimalnoj mekoj RISC-V jezgri, zajedno sa poređenjem sa simulatorom, ostaje slabije pokriveno. Time je opravdan pristup koji se razrađuje u narednim poglavljima.

Poglavlje 2

Hardverska platforma i FreeRTOS port

U ovom poglavlju opisuje se osnova na kojoj počiva cijeli rad. Najprije se predstavlja razvojna ploča i meki procesor na kojem se izvršava program, zatim prilagođeni port operativnog sistema FreeRTOS za taj procesor, te način na koji su organizovani memorija i ulazno-izlazni registri sistema na čipu. Na kraju poglavlja prikazuje se osnovni dokaz da operativni sistem zaista radi na hardveru, jer bez te potvrde nijedan kasniji rezultat ne bi imao smisla. Ovo poglavlje predstavlja temelj za razumijevanje kasnijih poglavlja u kojima se aplikacija razdvaja na zadatke i uklapa u stroge resurse ploče.

2.1 Tang Nano 9K i meki procesor PicoRV32

Razvojna ploča Tang Nano 9K zasnovana je na programabilnom logičkom polju proizvođača Gowin, modela GW1NR-9C. Ploča se taktuje vanjskim oscilatorom od 27 megaherca i raspolaže ograničenim skupom logičkih elemenata i blokovske memorije, što je kasnije ključno za uklapanje sistema. Na ploči je putem sinteze digitalnog sklopa realizovan meki procesor PicoRV32, jednostavna jezgra zasnovana na otvorenoj skupovnoj arhitekturi RISC-V [7]. Procesor je konfigurisan za cjelobrojni skup instrukcija sa množenjem, što se u alatima označava kao *rv32im*.

PicoRV32 je višeciklusna jezgra, što znači da svaka instrukcija troši više takta. Takva izvedba je mala i štedljiva po pitanju logike, ali sporija po izvršenoj instrukciji u odnosu na cjevovodne procesore. Ta osobina je važna za kasnije tumačenje izmjerenih vrijednosti, jer se broj takta po zamjeni konteksta ne može direktno tumačiti kao broj izvršenih instrukcija.

2.2 Prilagođeni port operativnog sistema FreeRTOS

FreeRTOS je operativni sistem za rad u realnom vremenu otvorenog koda, dostupan za veliki broj arhitektura [8]. Za potrebe ovog sistema korišten je prilagođeni port jezgre verzije V10.5.1 za PicoRV32. Raspoređivač periodično dobija takt iz vanjskog brojača koji generiše prekid na liniji broj pet, podesen na frekvenciju od sto herca. Promjena zadatka, odnosno zamjena konteksta, pokreće se programskim prekidom na liniji broj šest.

Važna posebnost ovog porta je obavezna upotreba prevodilačke opcije koja isključuje takozvanu relaksaciju, *-mno-relax*. Razlog je taj što PicoRV32 u korištenoj konfiguraciji pri ulasku u prekid može prepisati registre globalnog i nitnog pokazivača, pa se program ne smije oslanjati na globalni pokazivač. Ova naizgled sitna postavka ključna je za ispravan rad sistema i predstavlja primjer pažnje koju zahtijeva rad na niskom nivou.

2.3 Memorijska i adresna organizacija sistema na čipu

Sistem na čipu razdvaja prostor programa i prostor podataka. Program se nalazi u zasebnoj memoriji, takozvanom programskom prozoru, dok se podaci i stog nalaze u radnoj memoriji veličine šesnaest kilobajta. Pristup periferni jedinicama ostvaruje se preslikavanjem u memoriju (*Memory-Mapped Input/Output*, MMIO), gdje svaka periferija zauzima svoj raspon adresa. Serijska jedinica, brojac, opšti ulazi i izlazi te video jedinica imaju odvojene adresne opsege. Pojednostavljena slika ove organizacije prikazana je na slici 2.1.



Slika 2.1: Pojednostavljena adresna i memorijska organizacija sistema na čipu.

Razdvajanje programa i podataka, kao i veličine pojedinih memorija, kasnije postaje središnji problem pri uklapanju cijele igre sa video izlazom, jer svaka od tih memorija troši ograničenu blokovsku memoriju ploče.

2.4 Dokaz pokretanja na hardveru

Prije bilo kakvog portanja aplikacije, potvrdjeno je da operativni sistem uopšte radi na ploči. Sintetizovan je sklop sa serijskim izlazom, a u njega je ugrađen jednostavan program sa tri zadatka. Nakon upisa sklopa u memoriju ploče, preko serijske veze na brzini od sto petnaest hiljada bita u sekundi dobijeni su očekivani ispisi zadataka i njihovo smjenjivanje. Time je pokazano da takt raspoređivača, programski prekid za zamjenu konteksta i međusobno isključivanje pristupa serijskoj jedinici rade ispravno.

2.5 Zaključak poglavlja

U poglavlju je predstavljena hardverska osnova rada. Opisani su ploča Tang Nano 9K i meki procesor PicoRV32, prilagođeni port operativnog sistema FreeRTOS sa taktom iz vanjskog brojaca i zamjenom konteksta putem programskog prekida, te organizacija memorije i perifernih jedinica. Potvrđeno je da sistem radi na stvarnom hardveru. Ova osnova omogućava da se u narednom poglavlju pristupi razdvajanju aplikacije na zasebne zadatke.

Poglavlje 3

Razdvajanje aplikacije na zadatke

U ovom poglavlju opisuje se kako je monolitna aplikacija TetriSaraj prevedena u skup zasebnih zadataka pod upravom raspoređivača. Najprije se objašnjava razdvajanje čiste logike igre od dijela koji pristupa hardveru, jer je takvo razdvajanje preduslov za rad pod operativnim sistemom. Zatim se opisuje arhitektura zadataka i načini njihove komunikacije. Na kraju se prikazuje dokaz da logika igre zaista napreduje pod raspoređivačem na stvarnom hardveru, i to bez video izlaza, kako bi se prvo potvrdila ispravnost samog toka igre. Ovaj korak je važan jer odvaja pitanje ispravnosti logike od kasnijeg pitanja uklapanja video izlaza.

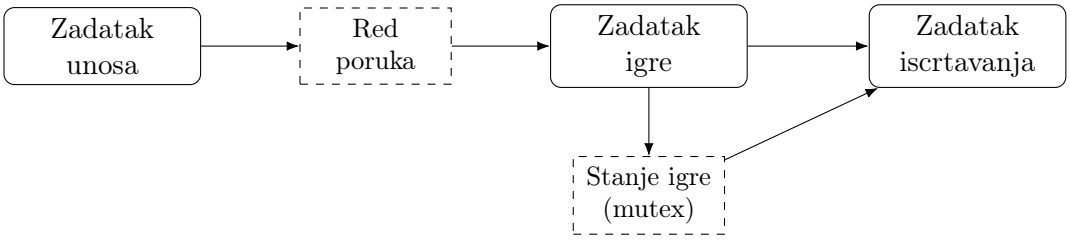
3.1 Razdvajanje logike igre i pristupa hardveru

Izvorna aplikacija držala je stanje igre u globalnim promjenljivim i direktno pristupala perifernim jedinicama unutar jedne velike petlje. Da bi se igra mogla izvršavati kao zadatak, kod je razdvojen na dva dijela. Prvi dio čini čista logika igre koja radi isključivo nad strukturom stanja u memoriji, bez ikakvog pristupa hardveru. Drugi dio čini sloj platforme koji obavlja pristup serijskoj jedinici, dugmadima i video memoriji. Takvo razdvajanje omogućava da se ista logika igre koristi i u inačici sa video izlazom i u inačici koja stanje ispisuje tekstualno preko serijske veze.

3.2 Arhitektura zadataka i komunikacija

Aplikacija je podijeljena na četiri zadatka različitih prioriteta. Zadatak za unos periodično očitava ulaz i šalje ga dalje. Zadatak igre prima posljednji unos i pomjera stanje igre naprijed na zadatom periodu. Zadatak za

iscrtavanje osvježava prikaz kada se stanje promijeni. Pomoćni zadatak periodično ispisuje stanje sistema preko serijske veze. Za prenos unosa koristi se red poruka, a za zaštitu zajedničkog stanja igre koristi se međusobno isključivanje. Ova organizacija prikazana je na slici 3.1.



Slika 3.1: Arhitektura zadataka: unos se prenosi redom poruka, a zajedničko stanje igre štiti se međusobnim isključivanjem.

Razdvajanje na zadatke različitih prioriteta omogućava da zadatak igre ima prednost nad iscrtavanjem, a da unos ima najveću prednost. Time se postavlja osnova za predvidljivo ponašanje sistema u realnom vremenu.

3.3 Dokaz rada na hardveru bez video izlaza

Da bi se ispravnost logike potvrdila prije složenog uklapanja video izlaza, napravljena je inačica koja stanje igre iscrtava tekstualno preko serijske veze, bez pristupa video memoriji. Takva inačica uklapa se u već provjereni sklop sa serijskim izlazom. Nakon upisa u ploču, preko serijske veze dobijeni su okviri stanja igre. U njima se vidi okvir polja, padajuća figura, te promjena rezultata kada se figura zaključa i pojavi nova. Skraćeni ispis prikazan je u listingu 3.1.

```
--- frame 0x0002  score=0x0000  lives=0x03
##########################################
            IIII    ##
##########################################
[hb] beat=0x0002 tick=0x0000019C score=0x0000 ...

--- frame 0x0003  score=0x0001  lives=0x03
 OOO                ##
  O                 ##
##########################################
```

Listing 3.1: Skraćeni serijski ispis tekstualne inačice: rezultat raste sa 0x0000 na 0x0001 nakon zaključavanja figure.

Ispis pokazuje da takt raspoređivača napreduje, da četiri zadatka sarađuju preko reda poruka i međusobnog isključivanja bez narušavanja ispisa, te da stvarna logika igre napreduje kroz pojavu, padanje, zaključavanje i bodovanje figura. Time je dokazano da je igra pod upravom operativnog sistema funkcionalna na hardveru.

3.4 Zaključak poglavlja

U poglavlju je prikazano razdvajanje aplikacije na čistu logiku i sloj platforme, te podjela na četiri zadatka koji komuniciraju redom poruka i međusobnim isključivanjem. Rad logike igre pod raspoređivačem potvrđen je na hardveru putem tekstualnog ispisa, čime je ispravnost toka igre odvojena od pitanja video izlaza. Time su stvoreni uslovi da se u narednom poglavlju riješi uklapanje cijele igre zajedno sa video izlazom.

Poglavlje 4

Uklapanje cijele igre sa video izlazom

U ovom poglavlju obrađuje se središnji inženjerski problem rada, a to je uklapanje cijele igre zajedno sa operativnim sistemom i video izlazom u stroge resurse ploče Tang Nano 9K. Najprije se opisuje problem koji se javio pri prvom pokušaju, zatim se analizira njegov stvarni uzrok kroz zauzeće blokovske memorije. Nakon toga prikazuju se dva rješenja koja su zajedno omogućila uklapanje, kao i jedno važno otkriće o ponašanju alata za sintezu. Na kraju se daje konačni rezultat uklapanja i prilagodba video inicijalizacije. Ovo poglavlje predstavlja težište tehničkog doprinosa rada.

4.1 Problem uklapanja

Pri prvom pokušaju spajanja procesora sa operativnim sistemom i video jedinicom u rezoluciji visoke definicije, sinteza nije uspjela. Procijenjeni broj logičkih elemenata iznosio je oko šesnaest hiljada, dok ploča raspolaže sa oko osam i po hiljada. Na prvi pogled činilo se da je problem u količini logike, međutim, dublja analiza pokazala je da je stvarni uzrok bio drugdje.

4.2 Analiza zauzeća blokovske memorije

Pregledom izvještaja sinteze utvrđeno je da je već sama inačica sa serijskim izlazom trošila svu raspoloživu blokovsku memoriju, odnosno svih dvadeset šest blokova. Od toga je programska memorija zauzimala šesnaest blokova, radna memorija osam, a serijska jedinica dva. Kako više nije bilo slobodne blokovske memorije, video jedinica je svoju znakovnu i mapnu memoriju

morala smjestiti u logičke elemente i registre, što je dovelo do prividne eksplozije broja logičkih elemenata. Drugim riječima, problem nije bio u logici, nego u nedostatku blokovske memorije.

Pri pokušaju smanjenja memorija uočeno je i važno svojstvo alata za sintezu. Alat pouzdano prepoznaje i smješta memoriju u blokovsku memoriju samo kada je njena dubina jednaka stepenu broja dva. Kada je programska memorija pokušana smanjiti na dubinu koja nije stepen broja dva, alat je memoriju počeo realizovati pomoću registara, što je dovelo do gotovo devetnaest hiljada registara i nove neuspjele sinteze. Ovo otkriće odredilo je smjer svih daljih optimizacija, jer su sve dubine memorija morale ostati stepeni broja dva.

4.3 Dva rješenja: manji program i manja video memorija

Prvo rješenje odnosi se na programsku memoriju. Utvrđeno je da je program povezivao i one funkcije operativnog sistema koje se uopšte ne koriste. Uz prevođenje sa razdvajanjem funkcija u zasebne sekcije i uklanjanjem nekorištenih sekcija pri povezivanju, veličina programa smanjena je sa oko dvadeset dva i po hiljade bajta na oko petnaest hiljada bajta. Time je program pao ispod granice od šesnaest kilobajta, pa programska memorija sada zauzima osam umjesto šesnaest blokova.

Drugo rješenje odnosi se na video memoriju. Umjesto rezolucije visoke definicije, korištena je niža rezolucija, čime se znakovna memorija smanjila sa dvanaest hiljada dvije stotine osamdeset osam na četiri hiljade devedeset šest riječi. Obje veličine ostale su stepeni broja dva, što je u skladu sa ranije opisanim svojstvom alata. Zajedno, ova dva rješenja oslobodila su dovoljno blokovske memorije da se cijeli sistem uklopi.

4.4 Rezultat uklapanja i prilagodba video inicijalizacije

Nakon primjene oba rješenja, sinteza je uspjela, a sistem se uklopio u resurse ploče sa određenom rezervom. Poređenje zauzeća resursa prije i poslije optimizacije dato je u tabeli 4.1.

Tablica 4.1: Zauzeće resursa kombinovanog sistema prije i poslije optimizacije.

Resurs	Prije	Poslije
Logički elementi	16041 (prekoračeno)	3348 / 8640
Registri	–	1349 / 6693
Blokovska memorija	26 / 26	22 / 26
programska (pROM)	16	8
radna (SP)	8	8
ostalo (SDPB)	2	6

Nakon što je sklop uklopljen, prilagođena je i video inicijalizacija u programu. Pošto je mapa polja ista u obje rezolucije, jer se polje sastoji od istog broja velikih znakova, mijenjala se samo veličina pojedinog znaka. Inicijalizacija znakova prilagođena je da upisuje znakove veličine šesnaest sa šesnaest tačaka umjesto ranijih dvadeset četiri sa trideset dvije. Sistem se nakon toga uspješno pokreće na hardveru i video inicijalizacija se izvršava do kraja, dok konačna vizuelna potvrda slike na ekranu ostaje stavka za daljnji rad.

4.5 Zaključak poglavlja

U poglavlju je riješen glavni inženjerski problem rada. Pokazano je da uzrok neuspjeha prvog uklapanja nije bila količina logike, nego nedostatak blokovske memorije, te da alat za sintezu pouzdano koristi blokovsku memoriju samo za dubine koje su stepeni broja dva. Smanjenjem programa uklanjanjem nekorištenih funkcija i korištenjem niže video rezolucije sistem je uklopljen u resurse ploče i pokrenut na hardveru. Time je pokazano da je cijela igra sa operativnim sistemom i video izlazom izvodljiva na ovoj niskobudžetnoj ploči.

Poglavlje 5

Mjerenje zamjene konteksta i poređenje sa simulatorom

U ovom poglavlju opisuje se mjerenje vremena zamjene konteksta operativnog sistema na stvarnom hardveru, te njegovo poređenje sa ranije dobijenim vrijednostima iz simulatora. Najprije se objašnjava metodologija mjerenja i razlog za određene odluke pri njenoj izradi. Zatim se prikazuju izmjereni rezultati na hardveru. Na kraju se ti rezultati pažljivo porede sa rezultatima iz simulatora, uz jasno isticanje ograničenja takvog poređenja. Cilj poglavlja je dati prvu mjerljivu karakterizaciju operativnog sistema na ovoj platformi, bez preuveličavanja zaključaka.

5.1 Metodologija mjerenja

Mjerenje oponasa test zamjene konteksta iz simulatorskog dijela istraživanja [6]. Jedna iteracija obuhvata dvije zamjene konteksta, odlazak iz prvog u drugi zadatak i povratak nazad. Vrijeme se mjeri slobodnim hardverskim brojačem koji radi na frekvenciji od dvadeset sedam megaherca. Za razliku od simulatorskog testa, koji koristi dva zadatka istog prioriteta i jedan semafor, čije ponašanje zavisi od dijeljenja vremena, ovdje su korištena dva semafora i zadatak odgovarač višeg prioriteta. Time svaka predaja semafora sigurno izaziva mjerenu zamjenu konteksta. Mjerena veličina je ista, broj takta za dvostruku zamjenu konteksta, a jednostruka zamjena dobija se dijeljenjem sa dva.

5.2 Rezultati na hardveru

Mjereno je pet stotina iteracija, a rezultat je bio ponovljiv kroz više upisa u ploču. Dobijene vrijednosti za dvostruku zamjenu konteksta iznose sedam hiljada šezdeset jedan takt kao najmanja, sedam hiljada sedamdeset devet kao prosječna i sedam hiljada devet stotina osamdeset dva kao najveća vrijednost. To odgovara prosječnom vremenu od oko dvije stotine šezdeset dvije mikrosekunde. Jednostruka zamjena konteksta iznosi oko tri hiljade pet stotina trideset takta, odnosno oko sto trideset jednu mikrosekundu. Mali raspon između najmanje i najveće vrijednosti pokazuje da nijedan takt raspoređivača nije upao u prozor mjerenja, pa su vrijednosti čiste.

Visok broj takta po zamjeni konteksta posljedica je toga što je PicoRV32 višeciklusna jezgra na relativno niskoj frekvenciji, kako je opisano u poglavlju 2. Zamjena konteksta pohranjuje i obnavlja cijeli skup cjelobrojnih registara, uz dodatni trošak rada sa semaforima.

5.3 Poređenje sa simulatorom

Rezultati iz simulatora, dobijeni emulacijom RISC-V sistema, daju za dvostruku zamjenu konteksta najmanju vrijednost od devet hiljada sto osamdeset osam takta, prosječnu od dvadeset tri hiljade sedam stotina četrdeset sedam i najveću od gotovo četiri miliona takta. Poređenje je dato u tabeli 5.1.

Tablica 5.1: Poređenje dvostruke zamjene konteksta na hardveru i u simulatoru (u taktovima).

Veličina	Hardver (27 MHz)	Simulator (25 MHz)
Najmanja	7061	9188
Prosječna	7079	23747
Najveća	7982	3895040

Poređenje treba tumačiti oprezno. Hardverska vrijednost dobijena je stvarnim brojačem takta, dok simulatorska vrijednost dolazi iz emuliranog brojača koji nije takovno tačan u odnosu na stvarni čip. Zbog toga se ne porede dva ista takta, nego model iz simulatora i stvarni hardver. Najpoštenija tačka poređenja je najmanja vrijednost, jer nije opterećena smetnjama, i tu su vrijednosti istog reda veličine. Prosječna i najveća vrijednost u simulatoru znatno su veće zbog kašnjenja koja unosi periodično čekanje u testu i takt raspoređivača. Zbog svega navedenog, ne iznosi se tvrdnja o

tačnom odnosu brzina, nego se zaključuje da je izmjerena hardverska zamjena konteksta po redu veličine u skladu sa čistom procjenom iz simulatora.

5.4 Zaključak poglavlja

U poglavlju je dato prvo mjerenje vremena zamjene konteksta operativnog sistema na stvarnom hardveru. Opisana je metodologija, prikazani su ponovljivi rezultati i izvršeno je oprezno poređenje sa simulatorom. Pokazano je da su čiste vrijednosti istog reda veličine, ali i da razlike u izvoru takta i uslovima mjerenja ne dopuštaju izvođenje tačnog odnosa brzina. Time je zaokružena mjerljiva karakterizacija sistema na ovoj platformi.

Poglavlje 6

Pravci daljnjeg istraživanja

U ovom poglavlju izdvajaju se mogući pravci daljnjeg istraživanja koji prirodno proizlaze iz urađenog rada. Svaki pravac povezan je sa konkretnom praznino m uočenom u radu ili u pregledu literature iz poglavlja 1. Pravci su poređani od onih koji najviše doprinose naučnoj vrijednosti rada prema onima koji predstavljaju šire proširenje. Cilj poglavlja je pokazati da postojeći rezultati čine osnovu za više zaokruženih istraživačkih cjelina.

6.1 Kvantifikacija troška operativnog sistema

Najvažniji pravec jeste mjerenje troška koji unosi operativni sistem u odnosu na golo izvršavanje. Ista operacija bi se izmjerila sa raspoređivačem i bez njega, čime bi se dobila stvarna cijena upotrebe operativnog sistema na ovoj mekoj jezgri. Ovaj pravec direktno odgovara na pitanje cijene koje je često u središtu literature o operativnim sistemima za rad u realnom vremenu [1].

6.2 Usklađivanje metrike i statistička obrada

Mjerenje iz poglavlja 5 obuhvata i trošak rada sa semaforima. Da bi se rezultat mogao poređivati sa literaturom, potrebno je dodatno izmjeriti užu, čistu zamjenu konteksta bez tog troška. Uz to, umjesto najmanje, prosječne i najveće vrijednosti, rezultate bi trebalo izraziti srednjom vrijednošću sa standardnom devijacijom i raspodjelom, te karakterizovati podrhtavanje. Time bi mjerenja dobila statističku strogost koju naučni rad zahtijeva.

6.3 Uticaj mikroarhitekture i skupa instrukcija

Meka jezgra dopušta promjenu konfiguracije, na primjer uključivanje sažetog skupa instrukcija ili sklopa za pomjeranje. Mjerenjem zamjene konteksta pri različitim konfiguracijama dobili bi se trendovi koji povezuju mikroarhitekturu sa cijenom operativnog sistema. Ovaj pravec je u skladu sa istraživanjima koja zajedno posmatraju mikroarhitekturu i operativni sistem [3].

6.4 Latencija i determinizam prekida pod opterećenjem

Pored zamjene konteksta, važna veličina je latencija prekida i njena postojanost pod različitim opterećenjem sistema. Mjerenjem latencije prekida i podrhtavanja, uz više zadataka i veće opterećenje, ocijenila bi se predvidljivost sistema, što je ključno svojstvo sistema za rad u realnom vremenu.

6.5 Sistematsko poređenje simulatora i hardvera

Pošto je emulator QEMU funkcijski tačan, a ne takovno tačan [5], zanimljivo je sistematski ispitati koliko dobro on predviđa vremenske veličine na stvarnom hardveru. Poređenjem više mjera na obje platforme dobio bi se sud o tome u kojim slučajevima je simulacija dovoljno vjerodostojna, a u kojima nije.

6.6 Hardversko ubrzanje raspoređivanja

Konačno, budući da je riječ o mekoj jezgri na programabilnoj logici, otvara se mogućnost dodavanja namjenskog sklopa za ubrzanje raspoređivanja ili zamjene konteksta. Takav pristup je u literaturi pokazao značajna ubrzanja [2], a ovaj rad pruža polaznu osnovu i mjerni okvir naspram kojeg bi se takvo ubrzanje moglo vrednovati.

6.7 Zaključak poglavlja

U poglavlju je izloženo šest pravaca daljnjeg istraživanja, od kvantifikacije troška operativnog sistema i statističke obrade mjerenja, preko uticaja mikroarhitek-

ture i ponašanja prekida, do sistematskog poređenja sa simulatorom i hardverskog ubrzanja raspoređivanja. Svaki pravec se naslanja na postojeće rezultate i prazninu u literaturi, čime se pokazuje da rad ima jasan potencijal za nastavak.

Zaključak

U radu je prikazano portanje aplikacije TetriSaraj sa golog izvršavanja na operativni sistem za rad u realnom vremenu FreeRTOS, na otvorenoj RISC-V platformi realizovanoj kao meki procesor PicoRV32 na ploči Tang Nano 9K. Na osnovu izloženog može se zaključiti da je sva tri pitanja postavljena u uvodu moguće potvrdno riješiti. Aplikacija je uspješno razdvojena na čiste zadatke pod upravom raspoređivača, a njen rad je dokazan na stvarnom hardveru putem serijskog ispisa u kojem logika igre napreduje. Cijeli sistem, zajedno sa video izlazom, uklopljen je u stroge resurse ploče i pokrenut na hardveru. Konačno, izmjereno je vrijeme zamjene konteksta na stvarnom hardveru i oprezno upoređeno sa rezultatima iz simulatora.

Posebno je istaknuto otkriće da uzrok neuspjelog uklapanja nije bila količina logike, nego nedostatak blokovske memorije, kao i to da alat za sintezu pouzdano koristi blokovsku memoriju samo za dubine koje su stepeni broja dva. Ta dva uvida bila su ključna za uspješno uklapanje sistema. Također je pokazano da su čiste vrijednosti zamjene konteksta na hardveru i u simulatoru istog reda veličine.

Iako rad obuhvata cijeli put od porta operativnog sistema do mjerenja na hardveru, pojedina pitanja ostaju otvorena za daljnje istraživanje. Vizuelna potvrda konačne slike igre na ekranu nije zatvorena u okviru ovog rada. Potpuno interaktivno upravljanje ograničeno je malim brojem fizičkih dugmadi na ploči. Također, mjerenja bi se mogla proširiti na trošak samog takta raspoređivača i na ponašanje sistema pod većim opterećenjem. Ovi pravci predstavljaju prirodan nastavak rada.

Popis slika

2.1 Pojednostavljena adresna i memorijska organizacija sistema na čipu. 12

3.1 Arhitektura zadataka: unos se prenosi redom poruka, a zajedničko stanje igre štiti se međusobnim isključivanjem. 14

Bibliografija

[1] UL Solutions. Measuring real-time operating system performance – part ii: Comparing FreeRTOS vs. Zephyr. Tehnički blog, `https://www.ul.com/sis/blog/measuring-real-time-operating-system-performance-part-ii-comparing-freertos-v` 2023. Pristupljeno 2026.

[2] Robert Balas, Alessandro Ottaviano, and Luca Benini. CV32RT: Enabling fast interrupt and context switching for RISC-V microcontrollers. *arXiv preprint arXiv:2311.08320*, 2023.

[3] Markus Scheck, Tammo Mürmann, and Andreas Koch. Co-exploration of RISC-V processor microarchitectures and FreeRTOS extensions for lower context-switch latency. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*, pages 394–408, 2026. doi: 10.1145/3779212.3790141.

[4] Marek Jahnke, Lucas Bublitz, and Ulf Kulau. Performance evaluation of PicoRV32 RISC-V softcore for resource-constrained devices. In *2023 IEEE Nordic Circuits and Systems Conference (NorCAS)*, pages 1–6, 2023. doi: 10.1109/NorCAS58970.2023.10305479.

[5] Fabrice Bellard and QEMU contributors. QEMU – A Generic and Open Source Machine Emulator. `https://www.qemu.org`, 2023. Funkcijski tačan emulator; pristupljeno 2026.

[6] Delaida Muminović. Kvantitativna analiza operativnog sistema FreeRTOS na arhitekturi RISC-V u okruženju QEMU. Prateći simulatorski dio istraživanja, rezultati testa zamjene konteksta, 2025.

[7] Clifford Wolf. PicoRV32 – A Size-Optimized RISC-V CPU. `https://github.com/YosysHQ/picorv32`, 2019. Pristupljeno 2026.

[8] Amazon Web Services. FreeRTOS – Real-Time Operating System for Microcontrollers. `https://www.freertos.org`, 2023. Verzija jezgre V10.5.1; pristupljeno 2026.